

Query Operators and Filtering

MongoDB provides powerful operators to filter and search documents in flexible ways.

In this section, you will learn how to use:

- Comparison operators (`$gt` , `$lt` , `$eq` , `$ne` , `$in` , `$nin`)
- Logical operators (`$or` , `$and` , `$not` , `$nor`)
- Array and embedded field queries
- Sorting and pagination

Note: All code examples are for MongoDB Compass shell (mongosh).

Comparison Operators

`$gt`, `$gte`, `$lt`, `$lte`

Find students older than 21:

```
db.students.find({ age: { $gt: 21 } })
```

Find students aged between 18 and 25:

```
db.students.find({ age: { $gte: 18, $lte: 25 } })
```

Find teachers with more than 5 years experience:

```
db.teachers.find({ experience: { $gt: 5 } })
```

\$eq, \$ne

Find students who are **not enrolled**:

```
db.students.find({ enrolled: { $ne: true } })
```

\$in, \$nin

Find students enrolled in either “MongoDB” or “Node.js”:

```
db.students.find({ course: { $in: ['MongoDB', 'Node.js'] } })
```

Find teachers NOT teaching Express or Python:

```
db.teachers.find({ subject: { $nin: ['Express', 'Python'] } })
```

Logical Operators

\$or

Find students enrolled in “Python” **or** age less than 20:

```
db.students.find({
  $or: [
    { course: 'Python' },
    { age: { $lt: 20 } }
  ]
})
```

\$and (default behavior)

Find students aged above 20 **and** enrolled:

```
db.students.find({
  age: { $gt: 20 },
  enrolled: true
})
```

You can also write it explicitly:

```
db.students.find({
  $and: [
    { age: { $gt: 20 } },
    { enrolled: true }
  ]
})
```

\$not

Find students **not** enrolled in "Node.js":

```
db.students.find({
  course: { $not: { $eq: 'Node.js' } }
})
```

Complex Query Example

Find enrolled students who are either: - Taking MongoDB with age > 21, OR - Taking Node.js with high grades (average > 90)

```
db.students.find({
  enrolled: true,
  $or: [
    { course: 'MongoDB', age: { $gt: 21 } },
    {
      course: 'Node.js',
      $expr: { $gt: [{ $avg: "$grades" }, 90] }
    }
  ]
})
```

```
]
})
```

Array Queries

Working with the `grades` array in our student documents:

Matching Array Elements

Find students who scored exactly 95 in any test:

```
db.students.find({ grades: 95 })
```

\$all - Multiple Values

Find students who scored both 90 and 95 at some point:

```
db.students.find({
  grades: { $all: [90, 95] }
})
```

\$size - Array Length

Find students with exactly 3 grades recorded:

```
db.students.find({
  grades: { $size: 3 }
})
```

\$elemMatch - Complex Array Conditions

Find students with at least one grade above 90:

```
db.students.find({
  grades: { $elemMatch: { $gt: 90 } }
})
```

Working with Average Grades

Find students with average grade above 85:

```
db.students.find({
  $expr: {
    $gt: [{ $avg: "$grades" }, 85]
  }
})
```

Sorting and Pagination

sort()

Sort students by age (descending):

```
db.students.find().sort({ age: -1 })
```

Sort by multiple fields:

```
// Sort by course (ascending), then by age (descending)
db.students.find().sort({ course: 1, age: -1 })
```

skip() and limit()

Get the first 3 students:

```
db.students.find().limit(3)
```

Implement pagination (skip first 2, then get next 3):

```
db.students.find().skip(2).limit(3)
```

Combined Example

Find top 3 performing MongoDB students:

```
db.students.find({  
  course: 'MongoDB'  
}).sort({  
  grades: -1  
}).limit(3)
```

Advanced Queries with Teachers

Join-like Queries

Find all students taught by Dr. Kumar:

```
// First, find Dr. Kumar's ID  
db.teachers.findOne({ name: 'Dr. Kumar' })  
  
// Then find all students with that teacherId  
db.students.find({  
  teacherId: ObjectId("507f1f77bcf86cd799439011")  
})
```

Count Operations

Count students per course:

```
db.students.countDocuments({ course: 'MongoDB' })
```

Count all enrolled students:

```
db.students.countDocuments({ enrolled: true })
```

Summary

- Use comparison operators (`$gt` , `$lt` , `$in` , etc.) for filtering
- Combine with logical operators (`$or` , `$and`) for complex queries
- Array operators (`$all` , `$size` , `$elemMatch`) for array fields
- Use `.sort()` , `.skip()` , and `.limit()` for result control
- All queries run directly in MongoDB Compass shell